



**UNIVERSIDAD AUTÓNOMA
DE AGUASCALIENTES**

**Reporte de proyecto final
ARDUINO UNO R3**

Mariana Isabel Alvarado de la Huerta

al217633@edu.uaa.mx

Danna Cristina Castro Hernández

dannacristinacastrohdz@gmail.com

Vanessa Carolina Torres Rojas

torres100vane124@gmail.com

César Andrés Zuleta Malanco

al300077@edu.uaa.mx

Electrónica

Luis Antonio Raygoza Pérez

19 de junio del 2026

Ingeniería en Sistemas Computacionales

Benemérita Universidad Autónoma de Aguascalientes

Tabla de contenido

1. Descripción del proyecto	3
2. Esquemático	4
2.1 Nota sobre la implementación física y simulada	4
2.2 Lista de Materiales del Sistema	4
2.3 Configuración de Pines del Microcontrolador	5
2.4 Matriz de Estados y Niveles de Operación.....	5
2.5 Esquemático del circuito	6
3. Código fuente.....	7
3.1 Código completo	7
3.2 Estructura general del programa	10
3.3 Inicialización del sistema (Setup).....	11
3.4 Adquisición de la señal analógica.....	11
3.5 Determinación de niveles de operación	11
3.6 Verificación del sistema de seguridad	11
3.7 Activación progresiva de cargas	12
3.8 Control del display de siete segmentos	12
3.9 Función de apagado del display	12
3.10 Temporización del sistema	12
4. Evidencia de funcionamiento.....	13
Conclusiones	17

1. Descripción del proyecto

Este proyecto consiste en un sistema automatizado de control y distribución progresiva de carga eléctrica por niveles, gobernado por un microcontrolador **Arduino Uno R3**. El sistema monitorea en tiempo real una señal analógica proveniente de un potenciómetro, la cual es procesada y segmentada en cuatro rangos operativos (Nivel 0 al Nivel 3).

De acuerdo con el rango detectado, el firmware activa de forma progresiva distintos actuadores de señalización y potencia, permitiendo una respuesta escalonada del sistema en función de la entrada analógica.

El funcionamiento general del circuito está condicionado por un mecanismo de seguridad e interbloqueo híbrido (hardware–software), compuesto por un interruptor deslizable (S2) y una compuerta lógica AND (U4, 74LS08). El comportamiento del sistema se divide en dos estados principales:

Estado de seguridad APAGADO (LOW)

Cuando el interruptor se encuentra en estado bajo, la compuerta AND bloquea la señal de habilitación, enviando un cero lógico al pin D5 del Arduino. En esta condición, el sistema se desactiva completamente: el LED indicador permanece apagado, el display de 7 segmentos se apaga y ninguna carga de potencia puede activarse, independientemente del valor del potenciómetro.

Estado de seguridad ACTIVADO (HIGH)

Cuando el interruptor está en estado alto, el sistema queda habilitado. El LED indicador se enciende y el sistema permite la lectura del potenciómetro, la visualización del nivel en el display y la activación progresiva de cargas de acuerdo con los siguientes niveles:

- **Nivel 0 (0–250 ADC):** Display en “0”. No se activan cargas.
- **Nivel 1 (251–500 ADC):** Display en “1”. Sin activación de cargas.
- **Nivel 2 (501–750 ADC):** Display en “2”. Se activa el ventilador (M1).
- **Nivel 3 (751–1023 ADC):** Display en “3”. Se activan ventilador (M1) y lámpara (L1).

2. Esquemático

El diseño electrónico del sistema se encuentra estructurado de manera modular en dos hojas de trabajo. La Hoja 1 concentra el nodo de control digital, la interfaz de adquisición analógica y el bloque de visualización, mientras que la Hoja 2 distribuye las etapas de acoplamiento de potencia y la conexión de las cargas externas de alta demanda.

2.1 Nota sobre la implementación física y simulada

Es importante señalar que existen diferencias entre la implementación física del proyecto y su representación en simulación.

En la **implementación física**, la carga luminosa corresponde a un **foco incandescente de 6.3V**, mientras que en el entorno de simulación este elemento se representa como una **lámpara (light bulb)** por disponibilidad de componentes en el software.

De manera similar, la carga inductiva utilizada físicamente corresponde a un **ventilador de corriente directa de 12V**, el cual en simulación se modela como un **motor de corriente directa (DC motor)**.

Ambas representaciones mantienen la misma función eléctrica equivalente (carga resistiva e inductiva respectivamente), por lo que el análisis del comportamiento del circuito no se ve afectado, únicamente cambia la nomenclatura del componente según el entorno utilizado.

2.2 Lista de Materiales del Sistema

Para la implementación completa del circuito simulado se utilizaron los componentes descritos en la siguiente tabla de materiales:

Referencia	Cantidad	Componente	Función
U1	1	Arduino Uno R3	Unidad de control principal
T1, T2	2	Transistor NPN 2N2222A	Conmutación de cargas
D1	1	Diodo 1N4007	Protección contra picos inductivos
D2	1	LED rojo	Indicador de estado
R1–R10	8	Resistencias 220 Ω	Limitación de corriente
R7, R8	2	Resistencias 470 Ω	Control de base de transistores
R11	1	Resistencia 10 k Ω	Pull-down de seguridad

Rpot1	1	Potenciómetro 10 kΩ	Entrada analógica
Digit1	1	Display 7 segmentos (5161BS)	Visualización de nivel
U4	1	74LS08 (AND)	Lógica de seguridad
S2	1	Switch deslizante	Habilitación del sistema
L1	1	Carga luminosa (foco 6.3V / lámpara en simulación)	Salida de potencia
M1	1	Ventilador 12V / motor DC en simulación	Carga inductiva
P2	1	Fuente 12V 5A	Alimentación de potencia

Tabla 1. Materiales usados.

2.3 Configuración de Pines del Microcontrolador

La distribución y el direccionamiento de las señales lógicas y analógicas en los puertos del Arduino Uno R3 se rigieron bajo la siguiente matriz de conexiones

Pin	Tipo	Componente	Función
A0	Entrada analógica	Potenciómetro	Lectura de nivel
D2	Salida	LED indicador	Estado del sistema
D3	Salida	Base T2	Control motor
D4	Salida	Base T1	Control lámpara
D5	Entrada	74LS08	Habilitación
D6–D12	Salida	Display	Control segmentos

Tabla 2. Pines del Microcontrolador.

2.4 Matriz de Estados y Niveles de Operación

El comportamiento dinámico y acumulativo de las salidas del sistema se encuentra condicionado por la digitalización de la entrada analógica y el estado físico de la seguridad:

Seguridad	Nivel	ADC	Display	LED	Motor	Lámpara
LOW	—	0–1023	OFF	OFF	OFF	OFF

HIGH	0	0–250	0	ON	OFF	OFF
HIGH	1	251–500	1	ON	OFF	OFF
HIGH	2	501–750	2	ON	ON	OFF
HIGH	3	751–1023	3	ON	ON	ON

Tabla 3. Niveles de Operación.

2.5 Esquemático del circuito

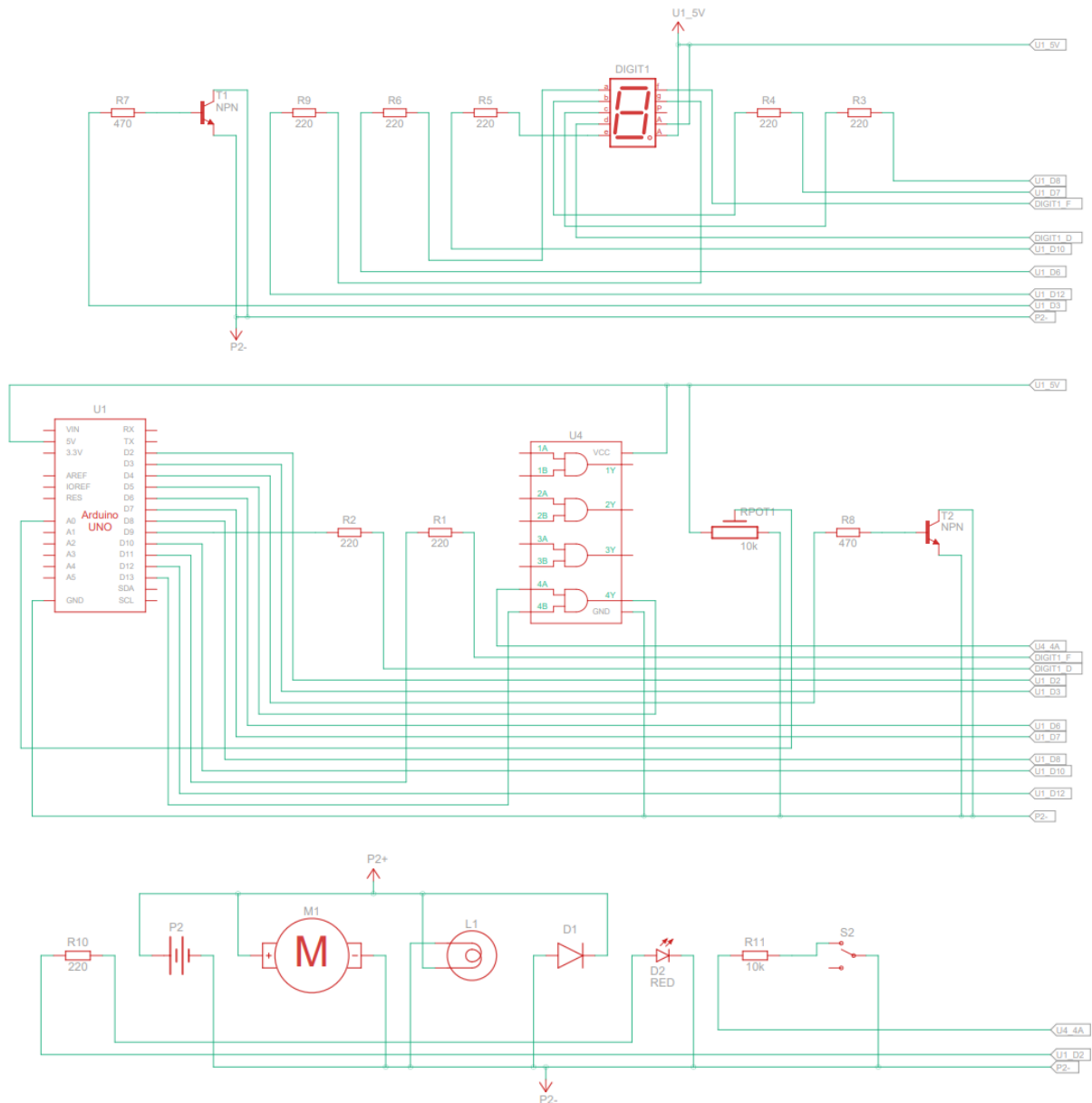


Figura 1. Esquemático del circuito.

3. Código fuente

3.1 Código completo

```
6 // Entradas
7 const int pot = A0;
8 const int enable = 5;
9
10 // Salidas
11 const int led = 2;
12 const int ventilador = 3;
13 const int foco = 4;
14
15 // Display 7 segmentos
16 const int segA = 6;
17 const int segB = 7;
18 const int segC = 8;
19 const int segD = 9;
20 const int segE = 10;
21 const int segF = 11;
22 const int segG = 12;
23
24 void setup() {
25
26     pinMode(led, OUTPUT);
27     pinMode(ventilador, OUTPUT);
28     pinMode(foco, OUTPUT);
29
30     pinMode(enable, INPUT);
31
32     for (int i = 6; i <= 12; i++) {
33         pinMode(i, OUTPUT);
34     }
35
36     apagarDisplay();
37 }
```

```
38
39 void loop() {
40
41     int valor = analogRead(pot);
42
43     int nivel = 0;
44
45     if (valor <= 250) {
46         nivel = 0;
47     }
48     else if (valor <= 500) {
49         nivel = 1;
50     }
51     else if (valor <= 750) {
52         nivel = 2;
53     }
54     else {
55         nivel = 3;
56     }
57
58     int seguro = digitalRead(enable);
59
60     if (seguro == HIGH) {
61
62         mostrarNumero(nivel);
63
64         // LED indicador
65         if (nivel >= 1)
66             digitalWrite(led, HIGH);
67         else
68             digitalWrite(led, LOW);
69
70         // Ventilador
71         if (nivel >= 2)
72             digitalWrite(ventilador, HIGH);
73         else
74             digitalWrite(ventilador, LOW);
75
```



```
76 // Foco
77 if (nivel >= 3)
78     digitalWrite(foco, HIGH);
79 else
80     digitalWrite(foco, LOW);
81
82 }
83 else {
84
85     digitalWrite(led, LOW);
86     digitalWrite(ventilador, LOW);
87     digitalWrite(foco, LOW);
88
89     apagarDisplay();
90 }
91
92 delay(100);
93 }
94
95 // =====
96 // DISPLAY ÁNODO COMÚN
97 // LOW = ENCENDIDO
98 // HIGH = APAGADO
99 // =====
100
101 void apagarDisplay() {
102
103     for (int i = 6; i <= 12; i++) {
104         digitalWrite(i, HIGH);
105     }
106 }
```

```

108 void mostrarNumero(int n) {
109
110     apagarDisplay();
111
112     switch (n) {
113
114         case 0:
115             digitalWrite(segA, LOW);
116             digitalWrite(segB, LOW);
117             digitalWrite(segC, LOW);
118             digitalWrite(segD, LOW);
119             digitalWrite(segE, LOW);
120             digitalWrite(segF, LOW);
121             break;
122
123         case 1:
124             digitalWrite(segB, LOW);
125             digitalWrite(segC, LOW);
126             break;
127
128         case 2:
129             digitalWrite(segA, LOW);
130             digitalWrite(segB, LOW);
131             digitalWrite(segD, LOW);
132             digitalWrite(segE, LOW);
133             digitalWrite(segG, LOW);
134             break;
135
136         case 3:
137             digitalWrite(segA, LOW);
138             digitalWrite(segB, LOW);
139             digitalWrite(segC, LOW);
140             digitalWrite(segD, LOW);
141             digitalWrite(segG, LOW);
142             break;
143     }
144 }

```

3.2 Estructura general del programa

El firmware fue desarrollado en lenguaje C++ utilizando Arduino IDE. El programa se encuentra organizado en bloques funcionales que facilitan la lectura, mantenimiento y escalabilidad del sistema.

Inicialmente se declaran las constantes asociadas a las entradas y salidas del circuito. Se asigna el pin analógico A0 para la lectura del potenciómetro, el pin D5 para la señal de habilitación proveniente de la compuerta lógica AND y diversos pines digitales para el control del LED indicador, el ventilador, el foco y los segmentos del display de siete segmentos.

3.3 Inicialización del sistema (Setup)

La función `setup()` se ejecuta una sola vez al encender el microcontrolador. En esta sección se configuran todos los pines de salida mediante la instrucción `pinMode()` y se establece el pin de habilitación como entrada digital.

Posteriormente, se ejecuta la función `apagarDisplay()`, garantizando que el display permanezca apagado durante el arranque del sistema y evitando estados indeterminados en la visualización.

3.4 Adquisición de la señal analógica

Durante cada ciclo de ejecución, el programa obtiene el valor del potenciómetro mediante la función `analogRead(pot)`, la cual devuelve un valor digital comprendido entre 0 y 1023, proporcional al voltaje aplicado en el pin A0.

Esta lectura es utilizada para determinar el nivel operativo del sistema.

3.5 Determinación de niveles de operación

La clasificación de niveles se realiza mediante una estructura condicional `if - else if`, que divide el rango total del convertidor analógico-digital en cuatro regiones:

- Nivel 0: 0 – 250
- Nivel 1: 251 – 500
- Nivel 2: 501 – 750
- Nivel 3: 751 – 1023

Esta segmentación permite transformar una señal analógica continua en estados discretos que posteriormente gobiernan la activación de las cargas.

3.6 Verificación del sistema de seguridad

Una vez determinado el nivel operativo, el programa consulta el estado de la señal de habilitación mediante la función `digitalRead(enable)`.

Si el interruptor de seguridad se encuentra activado y la compuerta lógica entrega un nivel alto (HIGH), el sistema continúa con la ejecución normal. En caso contrario, todas las salidas son desactivadas inmediatamente y el display es apagado.

Este mecanismo constituye una capa adicional de protección implementada tanto por hardware como por software.

3.7 Activación progresiva de cargas

Cuando la condición de seguridad es válida, el programa activa las salidas de manera acumulativa según el nivel calculado:

- Nivel 0: todas las cargas apagadas.
- Nivel 1: encendido del LED indicador.
- Nivel 2: encendido del LED y del ventilador.
- Nivel 3: encendido del LED, ventilador y foco.

Esta estrategia permite representar una demanda creciente de energía conforme aumenta el valor de entrada proporcionado por el usuario.

3.8 Control del display de siete segmentos

La visualización del nivel operativo se realiza mediante la función `mostrarNumero()`, la cual recibe como parámetro el nivel calculado.

Debido a que el display utilizado es de ánodo común, los segmentos se activan mediante lógica negativa. Por esta razón, un nivel LOW en un pin provoca el encendido del segmento correspondiente, mientras que un nivel HIGH lo apaga.

La selección de segmentos se realiza mediante una estructura switch, permitiendo representar los números del 0 al 3.

3.9 Función de apagado del display

La función `apagarDisplay()` recorre todos los pines asociados a los segmentos mediante un ciclo for, colocando cada uno en estado HIGH.

Esta rutina es utilizada tanto durante el arranque como cuando el sistema entra en estado de seguridad deshabilitado.

3.10 Temporización del sistema

Finalmente, al concluir cada iteración del ciclo principal, se ejecuta una pausa de 100 ms mediante la instrucción `delay(100)`.

Esta temporización reduce la velocidad de actualización del sistema, evita cambios excesivamente rápidos en las salidas y proporciona una operación más estable para el usuario.

4. Evidencia de funcionamiento

Con el objetivo de validar de manera práctica la correcta integración de las etapas de control, lógica digital y potencia, se sometió el sistema a una serie de pruebas funcionales y de laboratorio en tiempo real. El protocolo de evaluación consistió en registrar el comportamiento dinámico del circuito ante las variaciones de la señal analógica de entrada, comprobando tanto la respuesta visual como la activación secuencial de los actuadores.

Para garantizar la integridad de los componentes, el microcontrolador Arduino se alimentó mediante una conexión USB para la etapa de procesamiento, mientras que las cargas de mayor consumo (ventilador y foco) se energizaron a través de una fuente externa de potencia, consolidando una referencia de tierra común (GND) en todo el circuito.

A continuación, se presentan las evidencias fotográficas que demuestran el ensamblaje y comportamiento del sistema.

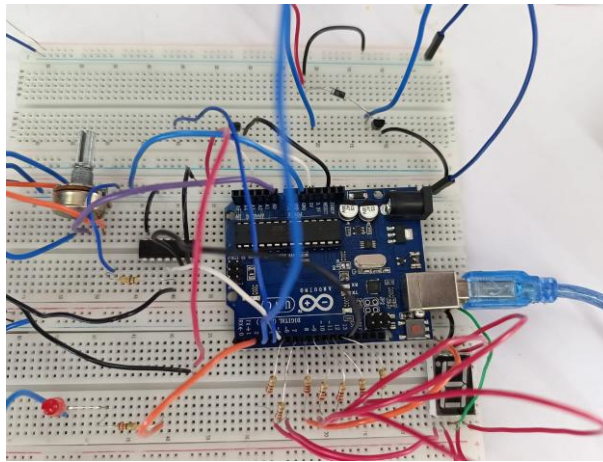


Figura 2. Ensamblaje del circuito.

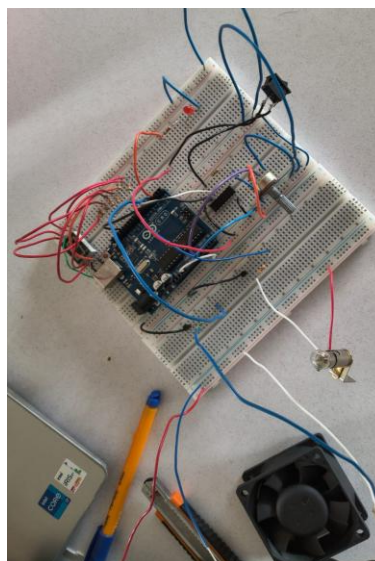


Figura 3. Ensamblaje del circuito con foco y ventilador.

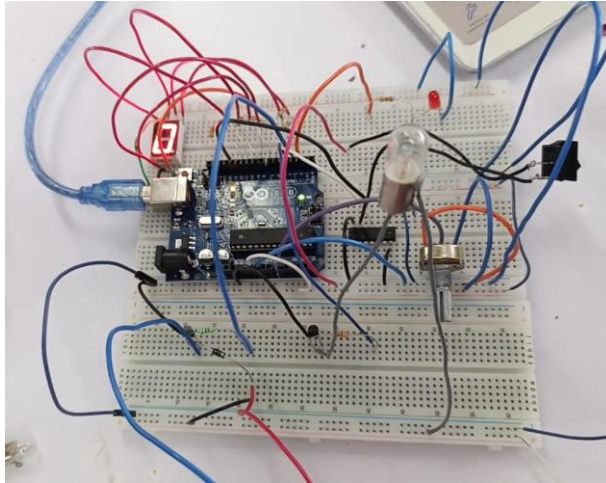


Figura 4. Circuito en Nivel 0 (estado de reposo).

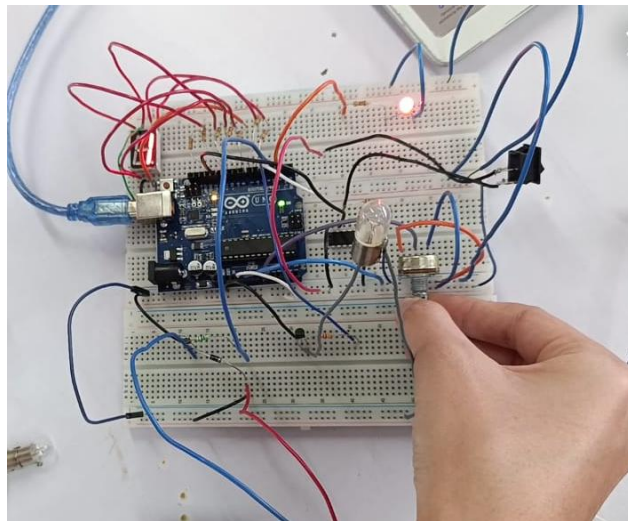


Figura 5. Circuito en Nivel 1.

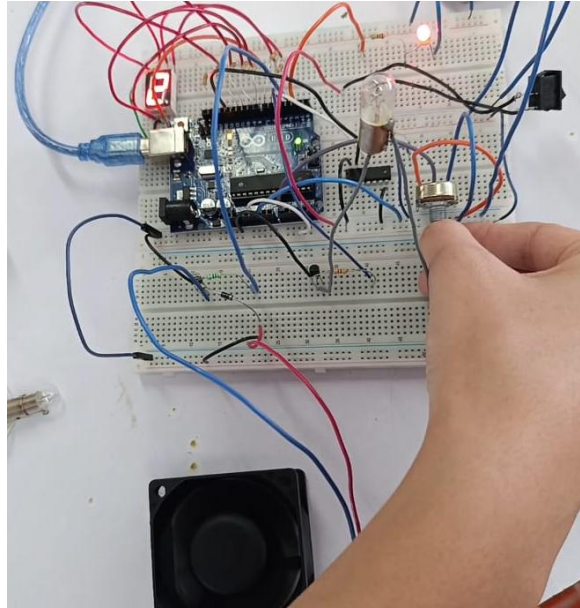


Figura 6. Circuito en Nivel 2.



Figura 7. Ventilador funcionando en Nivel 2.

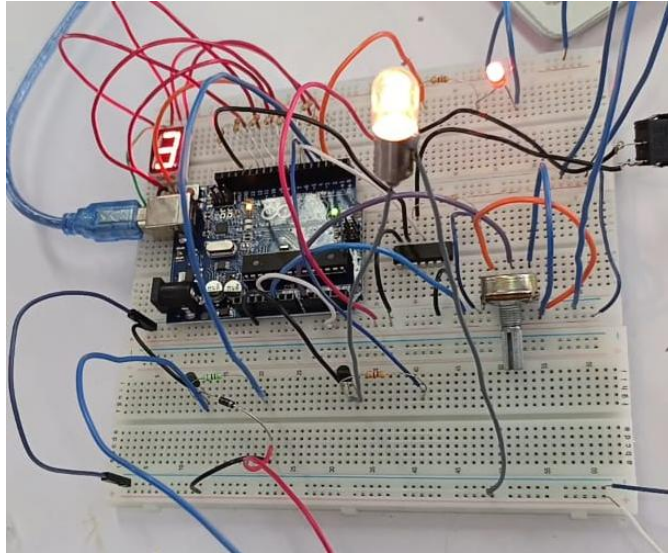


Figura 8. Circuito en Nivel 3.

Conclusiones

Mariana Isabel Alvarado de la Huerta

La realización de este proyecto me permitió comprobar de manera práctica cómo un microcontrolador como el Arduino puede interpretar variables del entorno mediante su ADC para clasificar un rango analógico en niveles operativos discretos. Asimismo, diseñar la etapa de potencia con transistores BJT 2N2222A me enseñó cómo una pequeña señal lógica digital de 5V es capaz de conmutar de forma segura cargas eléctricas externas de mayor consumo, como el ventilador y el foco de 12V.

Danna Cristina Castro Hernández

A través de este proyecto comprendí la importancia crítica de la protección y la estabilidad en los circuitos de control. Al trabajar con el ventilador de 12V, aprendí que las cargas inductivas generan picos de voltaje inverso destructivos al desenergizarse, y experimenté cómo la implementación en paralelo de un diodo de silicio 1N4007 absorbe estas corrientes transitorias para salvar al transistor y al Arduino de un daño permanente. Además, la integración del regulador 7805 me demostró cómo mantener una alimentación constante de 5V para la lógica digital a partir de una fuente de potencia mayor.

Vanessa Carolina Torres Rojas

El proyecto representó un excelente ejercicio para asimilar el concepto de redundancia y seguridad mediante hardware. Diseñar el módulo de habilitación con la compuerta lógica AND 74LS08 me ayudó a entender que el software no debe ser la única barrera de protección; la condición física del interruptor manual y la señal del Arduino deben coexistir para activar el sistema. Por otra parte, la codificación y el mapeo pin por pin para el display de siete segmentos enriquecieron mis habilidades para traducir estados lógicos internos en una interfaz visual directa para el usuario.

César Andrés Zuleta Malanco

La experiencia de realizar este proyecto me sirvió para entender las complejidades del acoplamiento físico real entre sistemas analógicos, digitales y de potencia. Más allá de las simulaciones ideales, las pruebas de laboratorio me enseñaron la importancia de establecer una referencia de tierra común (GND) para evitar comportamientos erráticos o caídas de tensión al activarse progresivamente el LED, el motor y el foco. Ver el circuito responder con precisión a las variaciones del potenciómetro consolidó mi aprendizaje sobre el comportamiento real de los semiconductores en un entorno integrado.